Final Laboratory Activity for Lab 3: Complete Role-Based Login System with Registration, Profiles & Admin Management

Course: Web Application Development
Module: REST APIs & Authentication Fundamentals
Name: Pauline Mae A. Gallamora
Course, Year & Section: BSIT 3-E

III. Pre-Lab Questions (10 pts)

Q1. Why does the server return a redirectTo value instead of letting the client decide where to send the user?

- **Server has the final say:** The server is the one that knows the user's real role, so it should be the one to say which page they go to.
- **Easy to change:** If a page moves or a new role is added, you only edit server.js once instead of editing every HTML file.

Q2. Since users are stored in memory, what happens when the server restarts? How do you fix this in a real app?

- **What happens:** The users array is reset to empty and the IDs start at 1 again, so every account is gone.
- **Fix:** Save users in a real database like MySQL, PostgreSQL or MongoDB so the data stays even after a restart.

Q3. Why does the login route return the same error for unknown email and wrong password?

- **Hides which emails exist:** If the server said "email not found" for one and "wrong password" for the other, someone could keep guessing emails and find out which ones are registered.
- **So:** A single message, "Invalid email or password.", tells an attacker nothing extra.

Q4. Why does sanitizeUser strip the password from every response?

- **No leaking:** The browser and anyone looking at the network tab would be able to see the passwords of users.
- **Not needed:** The page only needs the id, name, email and role, so there is no reason to send the password.

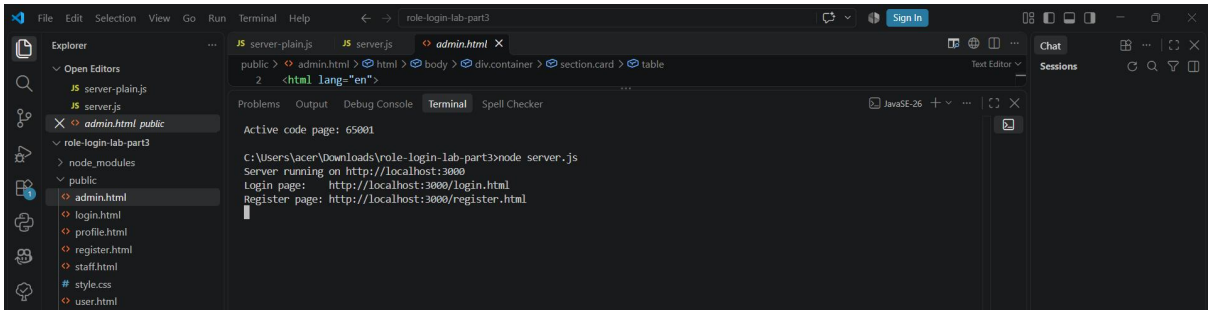
Q5. Is sessionStorage a strong form of protection? Why or why not?

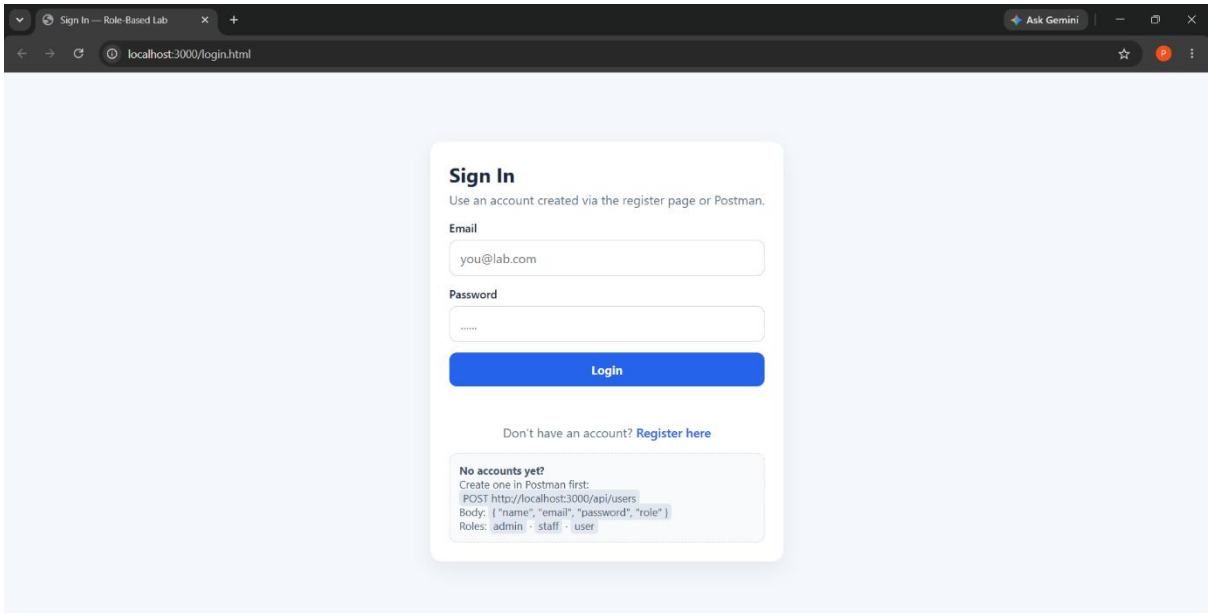
- **No, it is not strong.**
- **Why:** Anyone can open DevTools (F12) and edit sessionStorage, for example change the role from "user" to "admin". The browser also does not check it with the server, so it only hides pages, it does not truly secure them.
- **Better option:** Use server-side sessions or signed tokens that the server checks on every request.

VI. Laboratory Tasks

Task 1 — Run the Server (5 pts)

I ran node server.js and the console printed the three URLs. The login page loaded at http://localhost:3000/login.html.



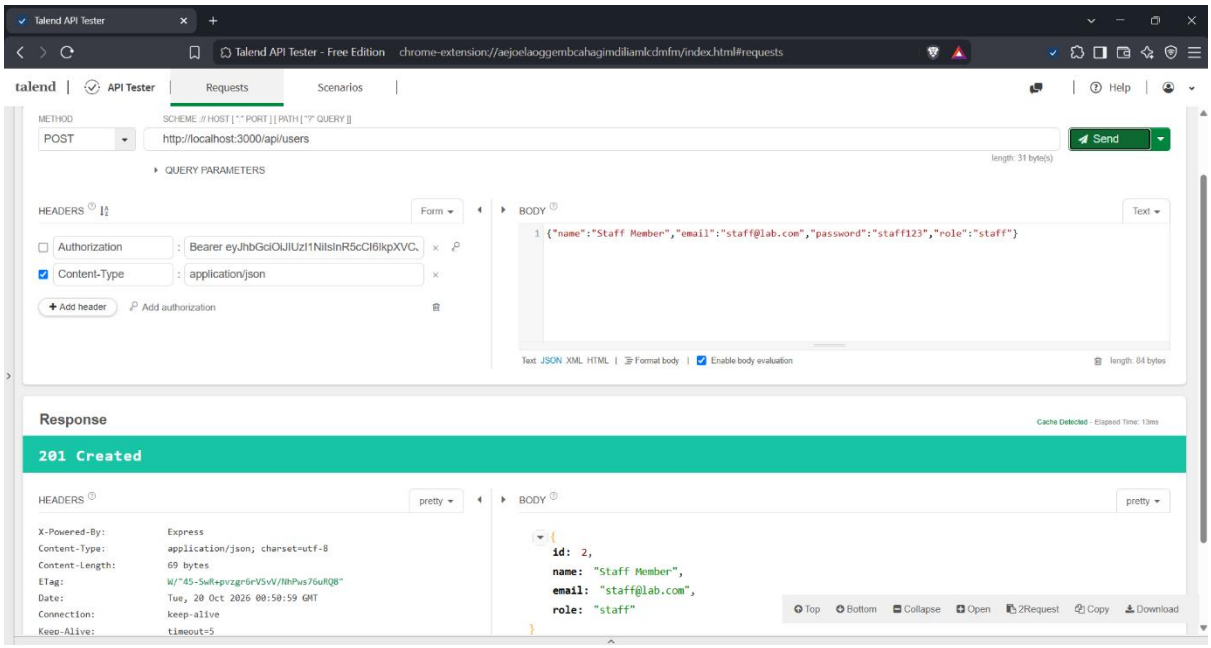


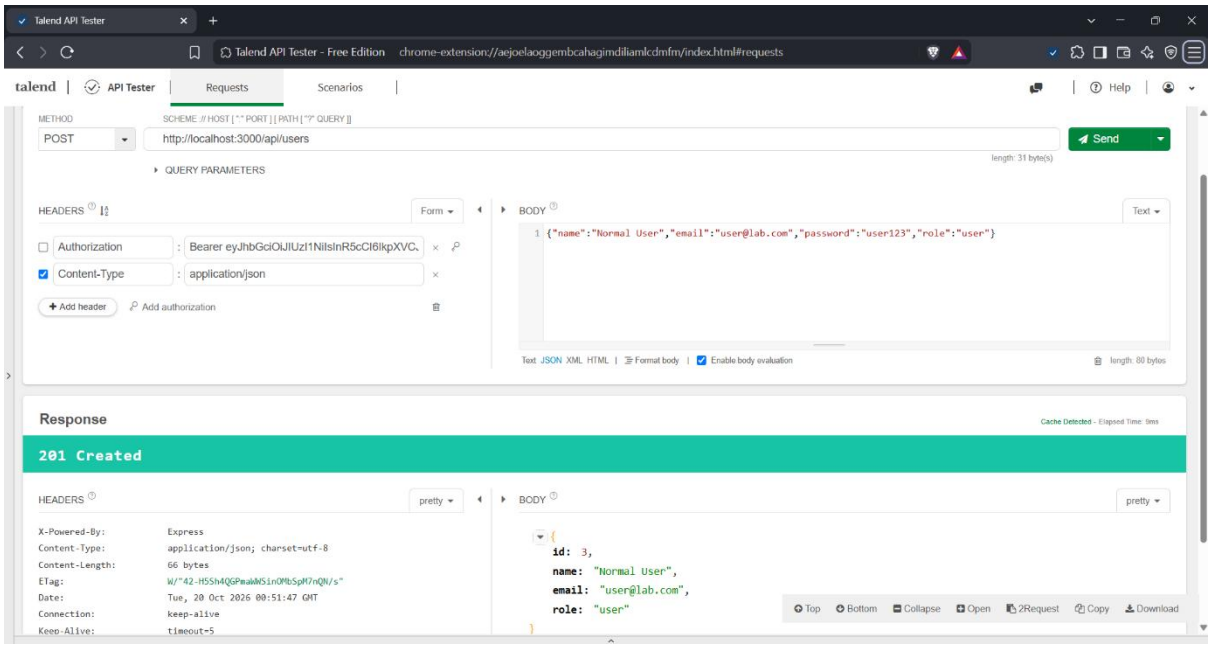
Deliverable: Screenshot of the terminal and login page.

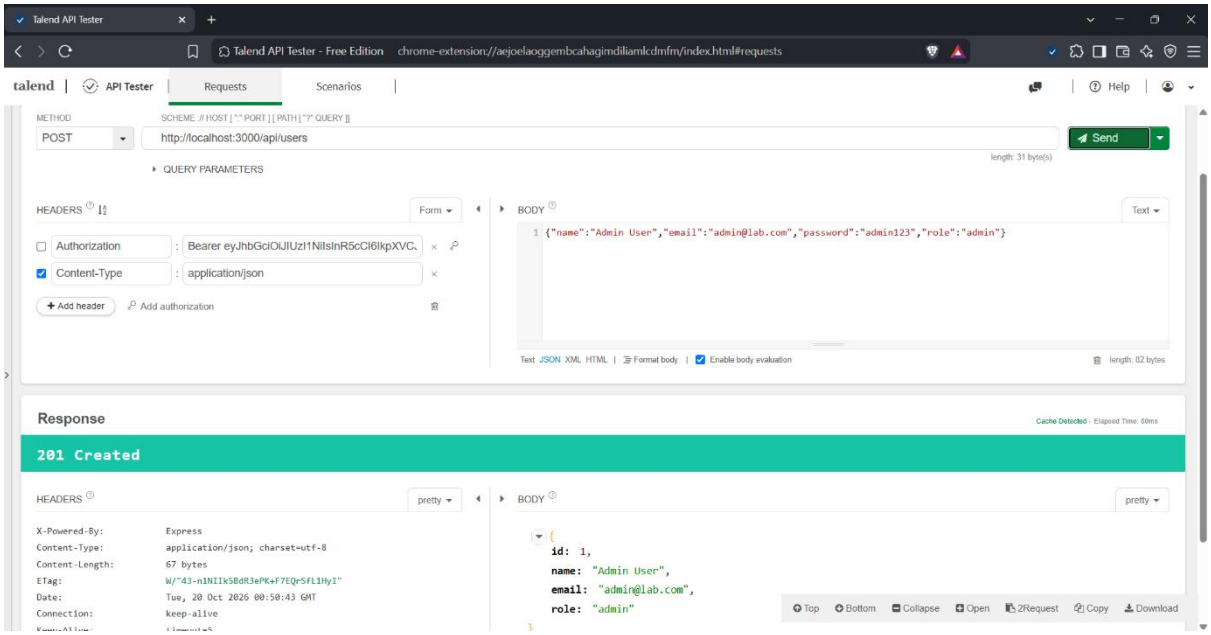
Task 2 — Create Users via Postman (10 pts)

I used Talend API Tester to POST one user per role to /api/users, then ran GET /api/users to confirm the passwords are not shown.

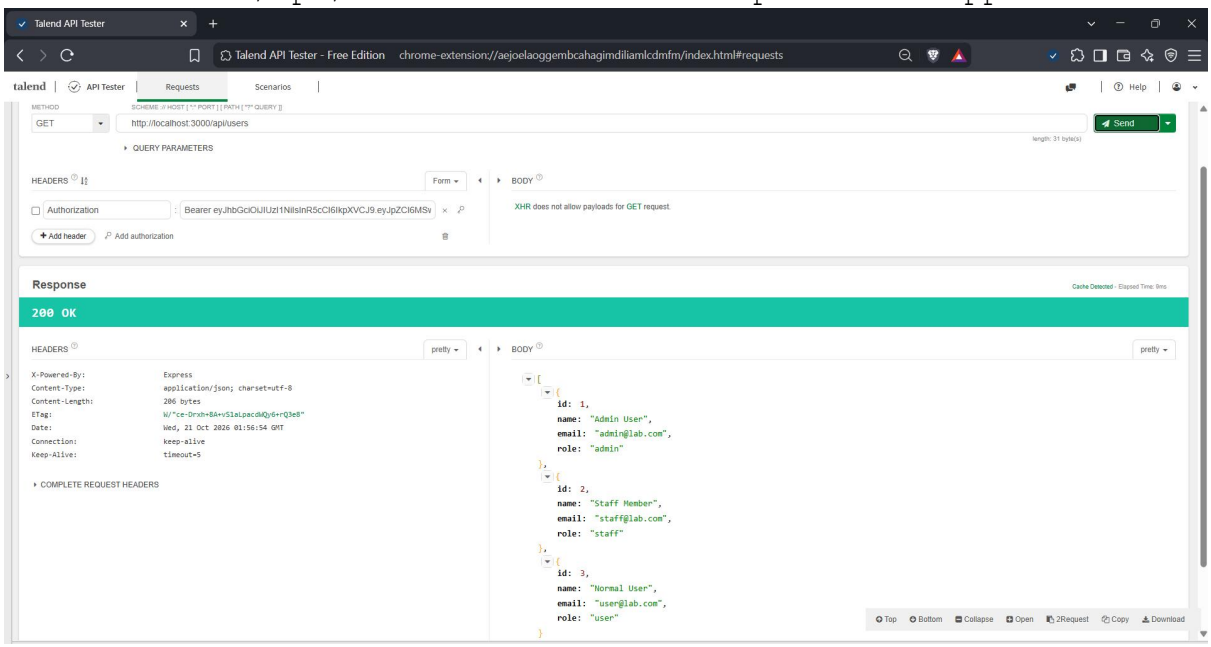
Role	Body used
admin	{"name":"Admin User","email":"admin@lab.com","password":"admin123","role":"admin"}
staff	{"name":"Staff Member","email":"staff@lab.com","password":"staff123","role":"staff"}
user	{"name":"Normal User","email":"user@lab.com","password":"user123","role":"user"}







Then run GET /api/users and confirm no passwords appear.

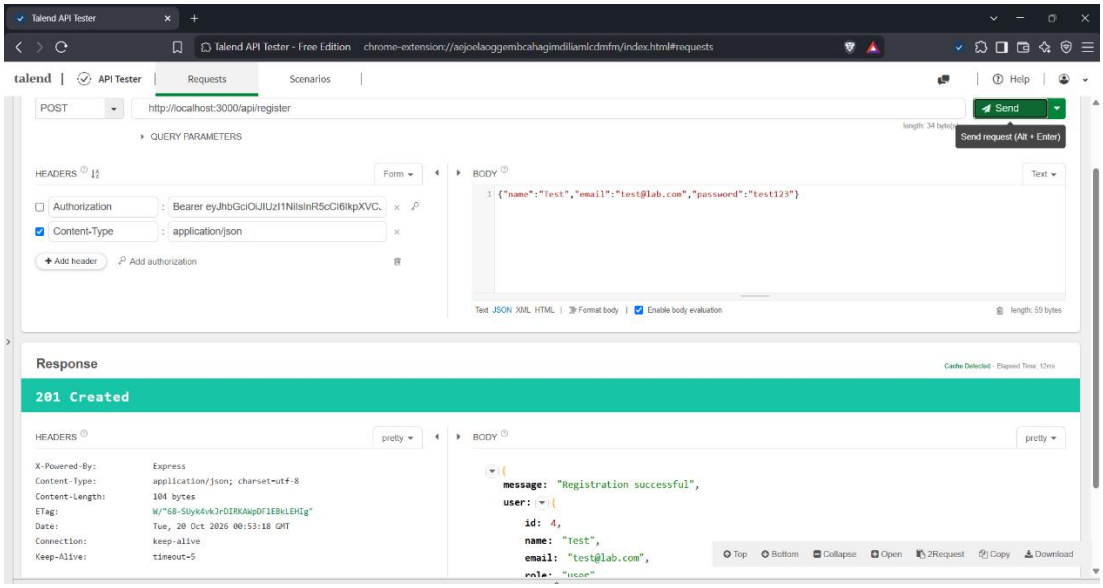


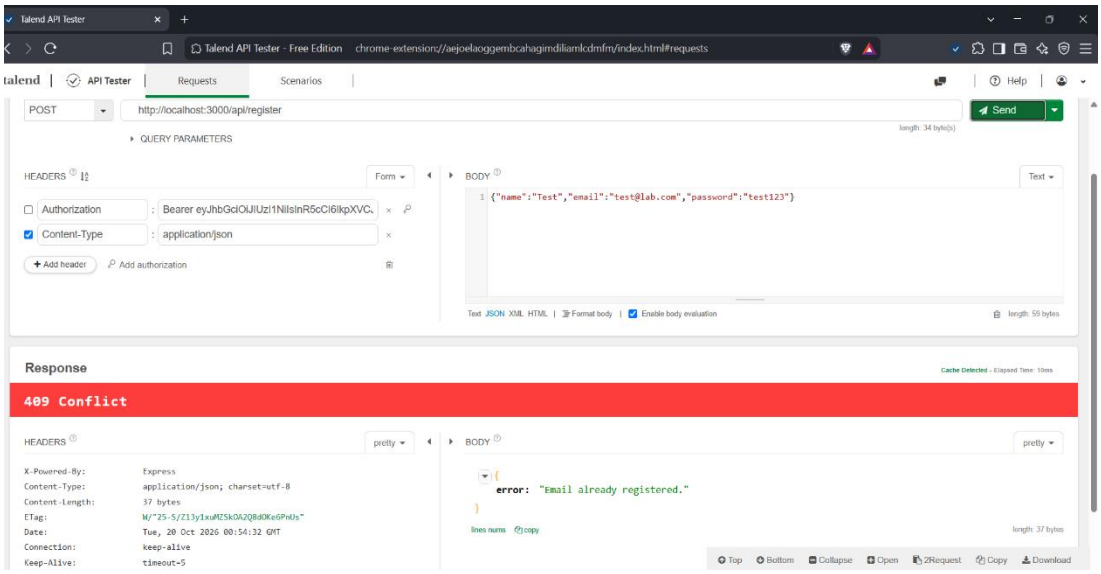
Deliverable: Screenshots of all 4 requests.

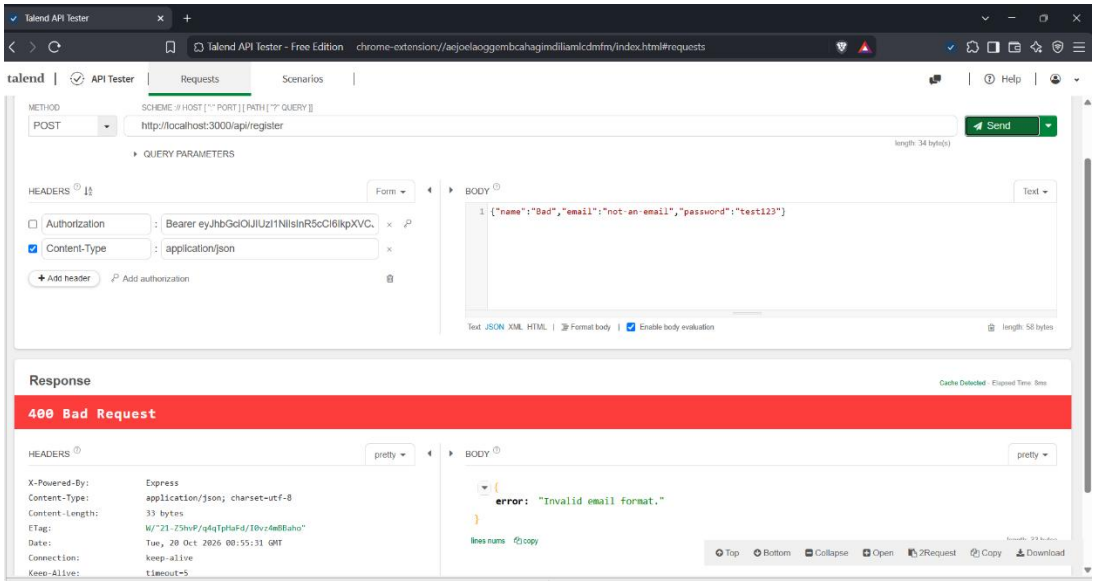
Task 3 — Test Registration in Postman (10 pts)

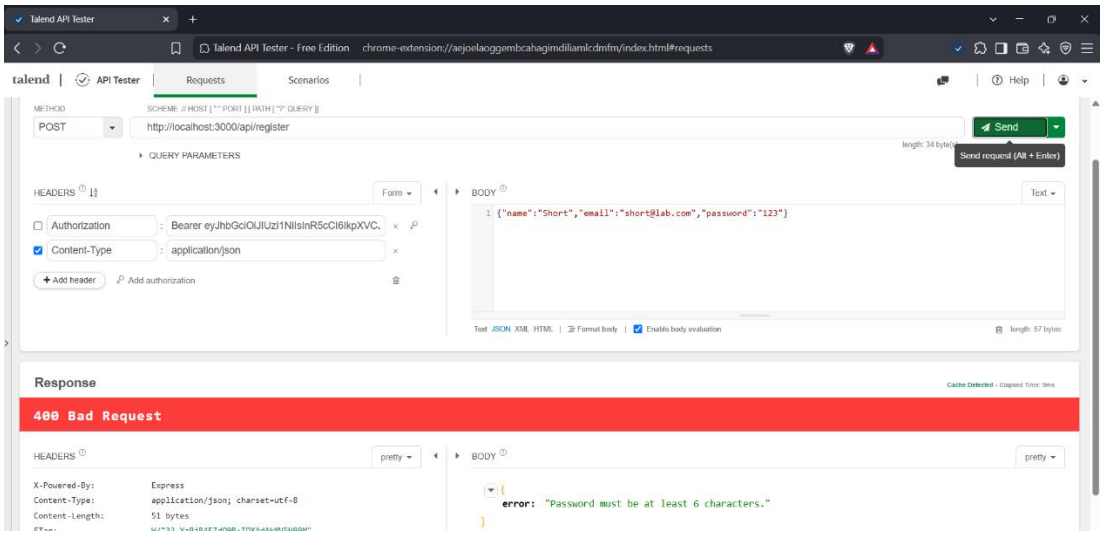
All five requests were sent to POST /api/register.

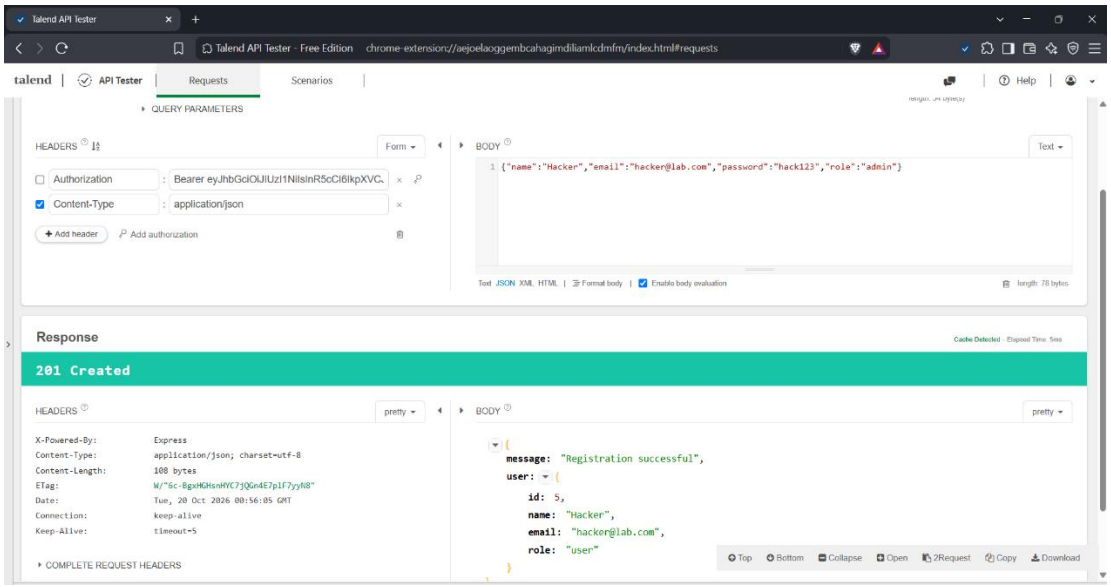
#	Body	Expected / Result
1	{"name":"Test","email":"test@lab.com","password":"test123"}	201 Created
2	Same email again	409 Conflict
3	{"name":"Bad","email":"not-an-email","password":"test123"}	400 Bad Request
4	{"name":"Short","email":"short@lab.com","password":"123"}	400 Bad Request
5	{"name":"Hacker","email":"hacker@lab.com","password":"hack123", "role":"admin"}	201, role forced to user









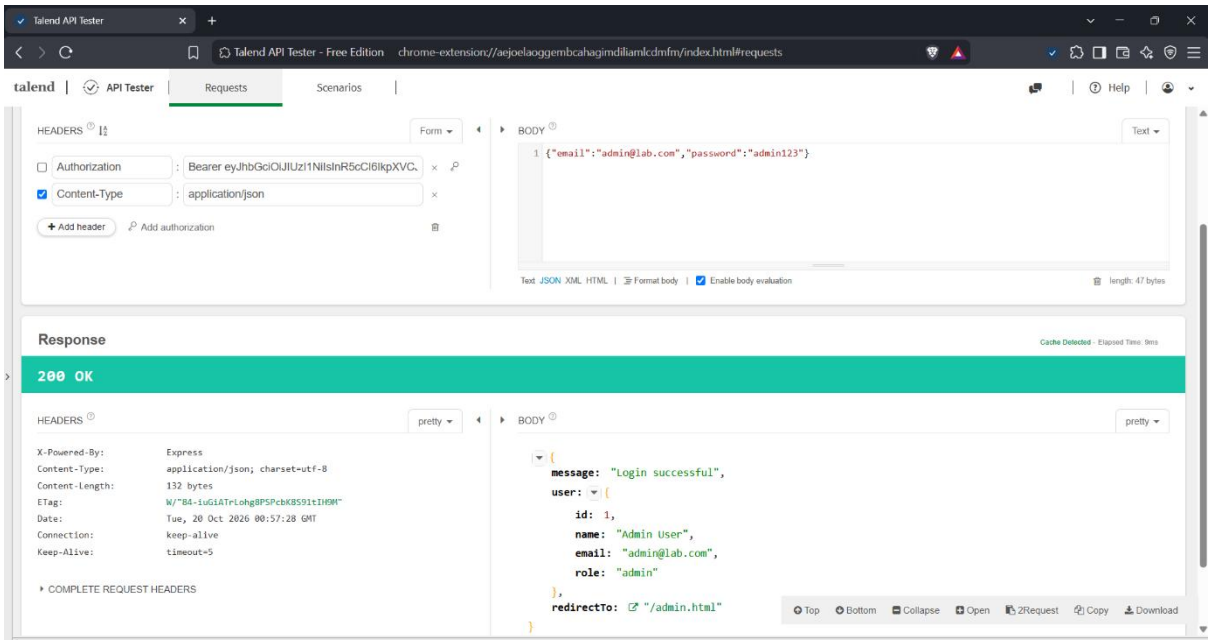


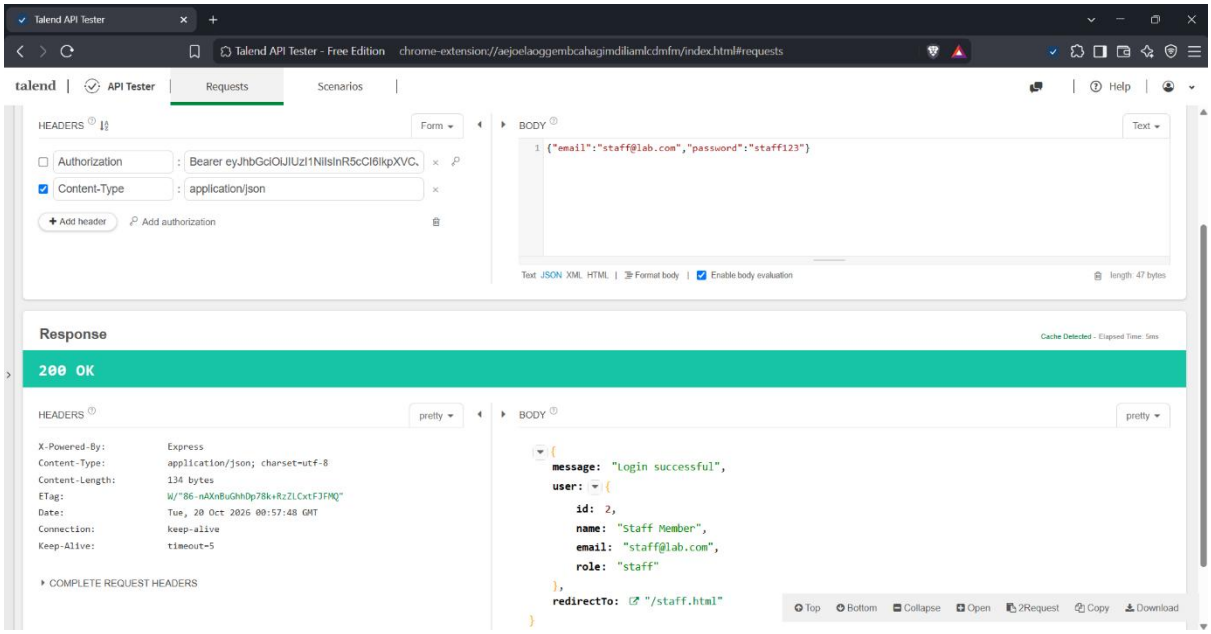
Deliverable: Screenshots of all 5 requests.

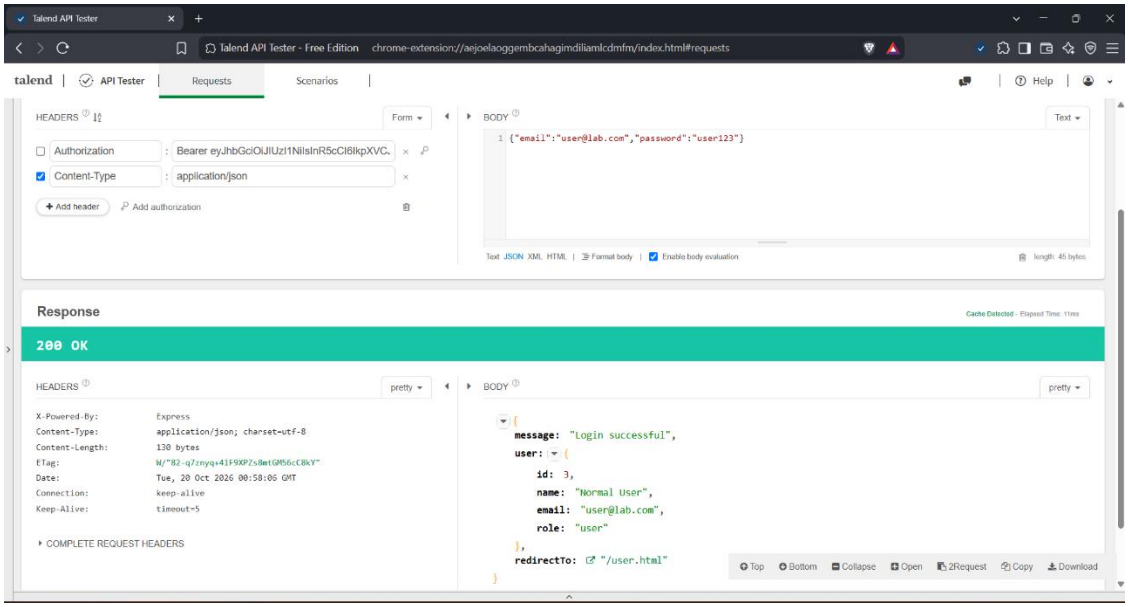
Task 4 — Test Login in Postman (10 pts)

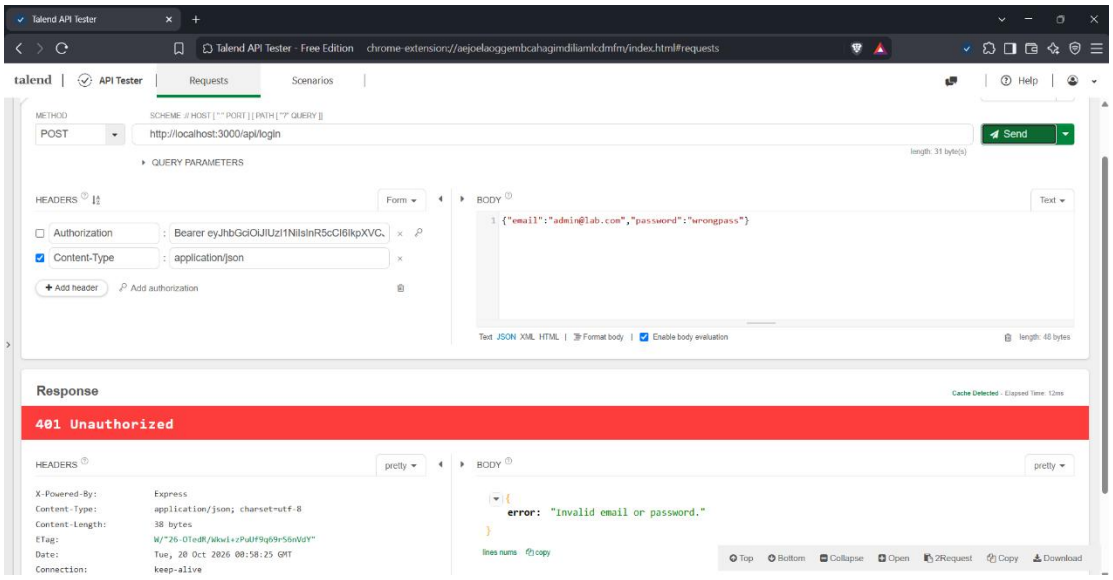
All six requests were sent to POST /api/login.

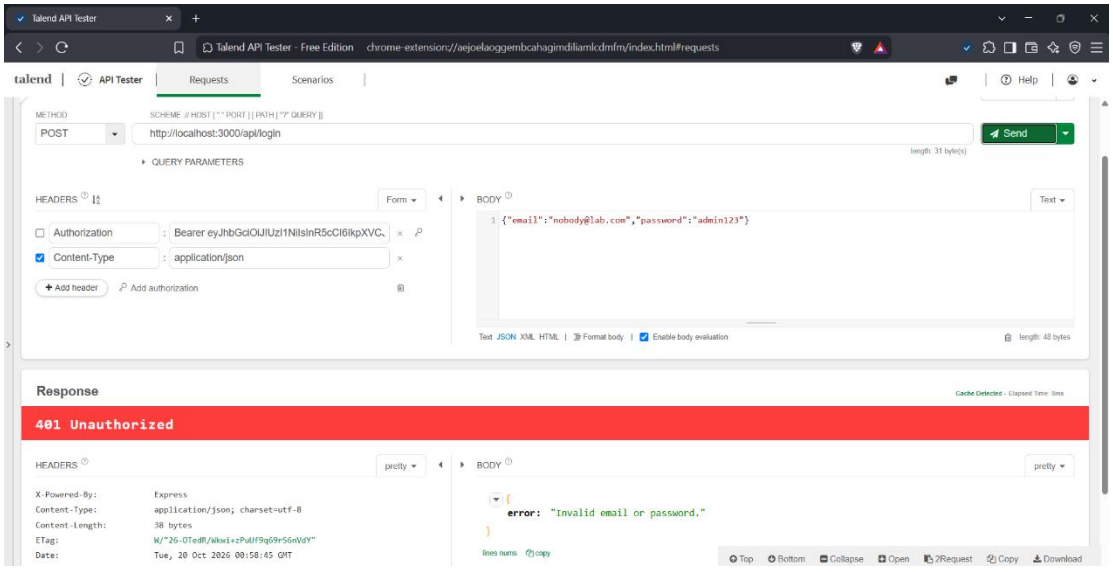
#	Case	Expected
1	Admin credentials	200, redirectTo: /admin.html
2	Staff credentials	200, redirectTo: /staff.html
3	User credentials	200, redirectTo: /user.html
4	Wrong password	401
5	Unknown email	401 (same error message)
6	Missing password	400

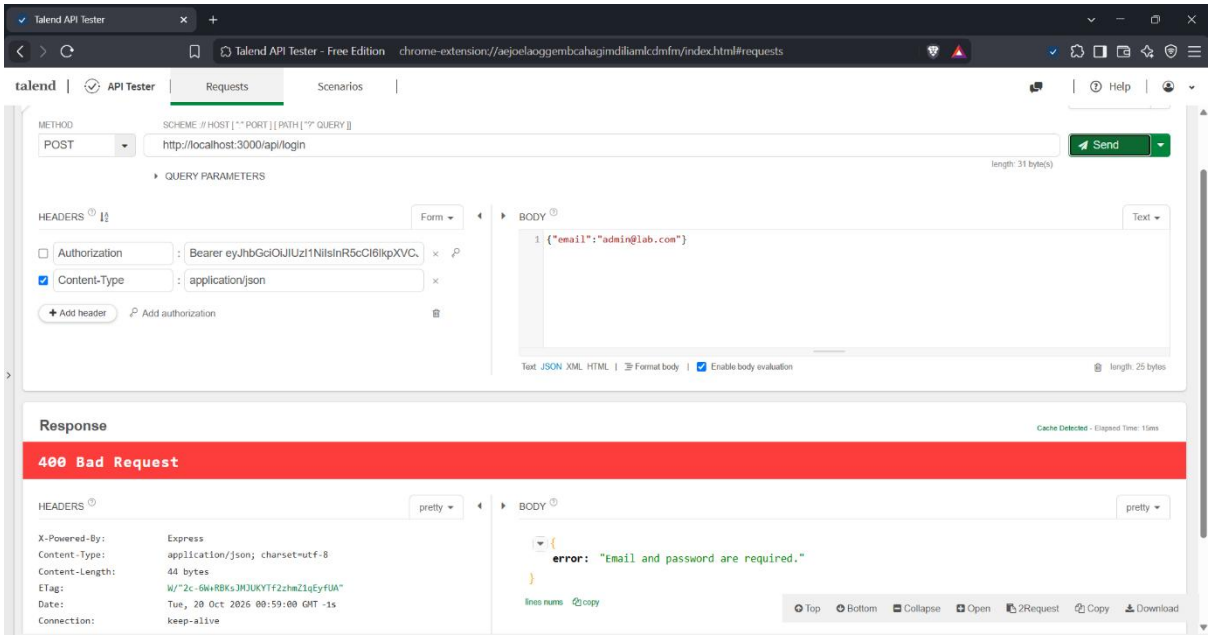





Talend API Tester
Talend API Tester - Free Edition
chrome-extension://aejoelaoggembcahagimdiliamlcdmfm/index.html#requests
talend | API Tester
Requests
Scenarios
Help
HEADERS
Form
BODY
Text
Authorization
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...
Content-Type
application/json
Add header
Add authorization
{"email":"user@lab.com","password":"user123"}
Text JSON XML HTML | Format body | Enable body evaluation
length: 45 bytes
Response
Cache Detected - Elapsed Time: 11ms
200 OK
HEADERS
pretty
BODY
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 130 bytes
ETag: W/"82-q7znyq+41F9XPZs8mtGM56cCBkY"
Date: Tue, 20 Oct 2026 00:58:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5
COMPLETE REQUEST HEADERS
{
message: "Login successful",
user: {
id: 3,
name: "Normal User",
email: "user@lab.com",
role: "user"
},
redirectTo: "/user.html"
}
Top Bottom Collapse Open 2Request Copy Download


Talend API Tester
Talend API Tester - Free Edition
chrome-extension://aejoelaoggembcahagimdiliamlcdmfm/index.html#requests
talend | API Tester
Requests
Scenarios
Help
METHOD
SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]
POST
http://localhost:3000/api/login
Send
length: 31 byte(s)
QUERY PARAMETERS
HEADERS
Form
BODY
Text
Authorization
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...
Content-Type
application/json
Add header
Add authorization
{"email":"admin@lab.com","password":"wrongpass"}
Text JSON XML HTML | Format body | Enable body evaluation
length: 48 bytes
Response
Cache Detected - Elapsed Time: 12ms
401 Unauthorized
HEADERS
pretty
BODY
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 38 bytes
ETag: W/"26-0TedR/WkwI+zPuUf9q69r56nVdY"
Date: Tue, 20 Oct 2026 00:58:25 GMT
Connection: keep-alive
{
error: "Invalid email or password."
}
lines nums copy
Top Bottom Collapse Open 2Request Copy Download

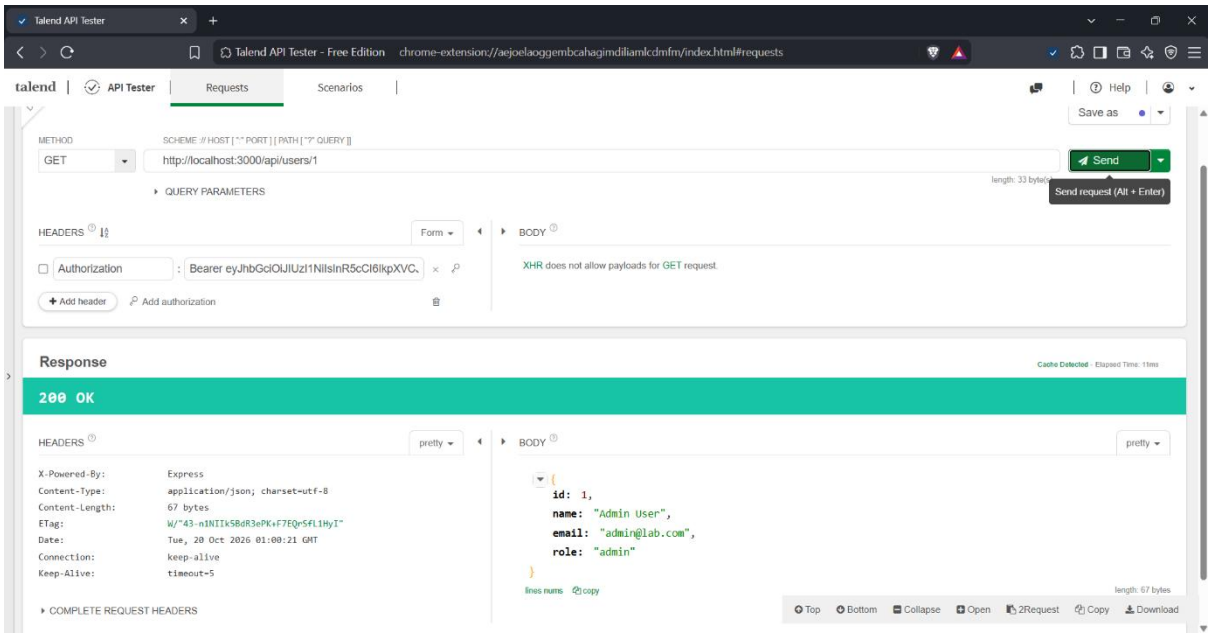

Talend API Tester
Talend API Tester - Free Edition
chrome-extension://aejoelaoggembcahagimdiliamlcdmfm/index.html#requests
talend | API Tester
Requests
Scenarios
Help
METHOD
SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]
POST
http://localhost:3000/api/login
Send
length: 31 byte(s)
QUERY PARAMETERS
HEADERS
Form
BODY
Text
Authorization
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...
Content-Type
application/json
Add header
Add authorization
{"email":"nobody@lab.com","password":"admin123"}
Text JSON XML HTML | Format body | Enable body evaluation
length: 48 bytes
Response
Cache Detected - Elapsed Time: 0ms
401 Unauthorized
HEADERS
pretty
BODY
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 38 bytes
ETag: W/"26-0TedR/WkwI+zPuUf9q69r56nVdY"
Date: Tue, 20 Oct 2026 00:58:45 GMT
{
error: "Invalid email or password."
}
lines nums copy
Top Bottom Collapse Open 2Request Copy Download

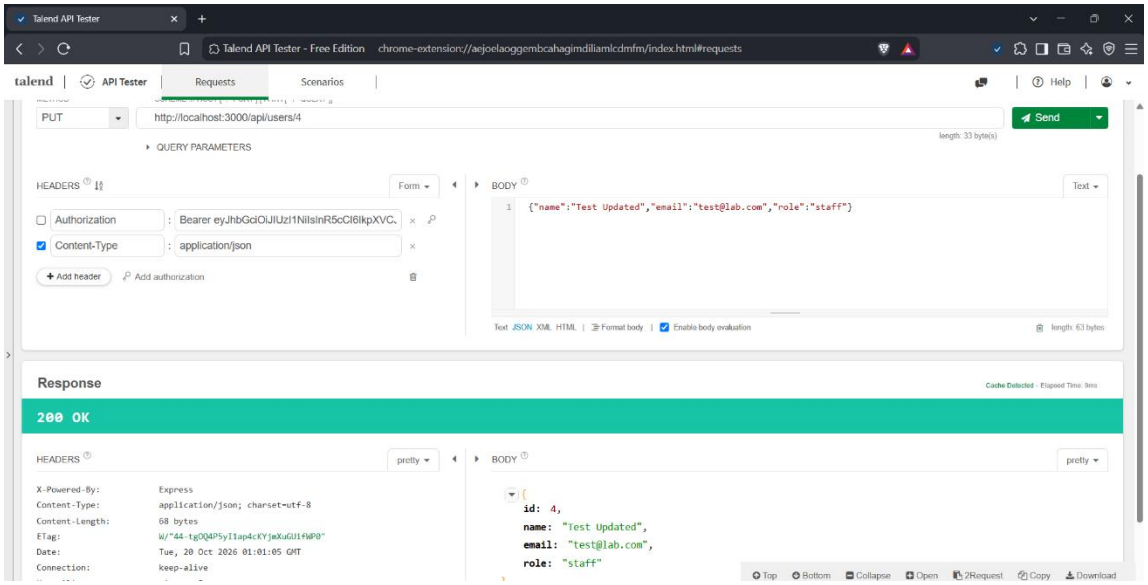


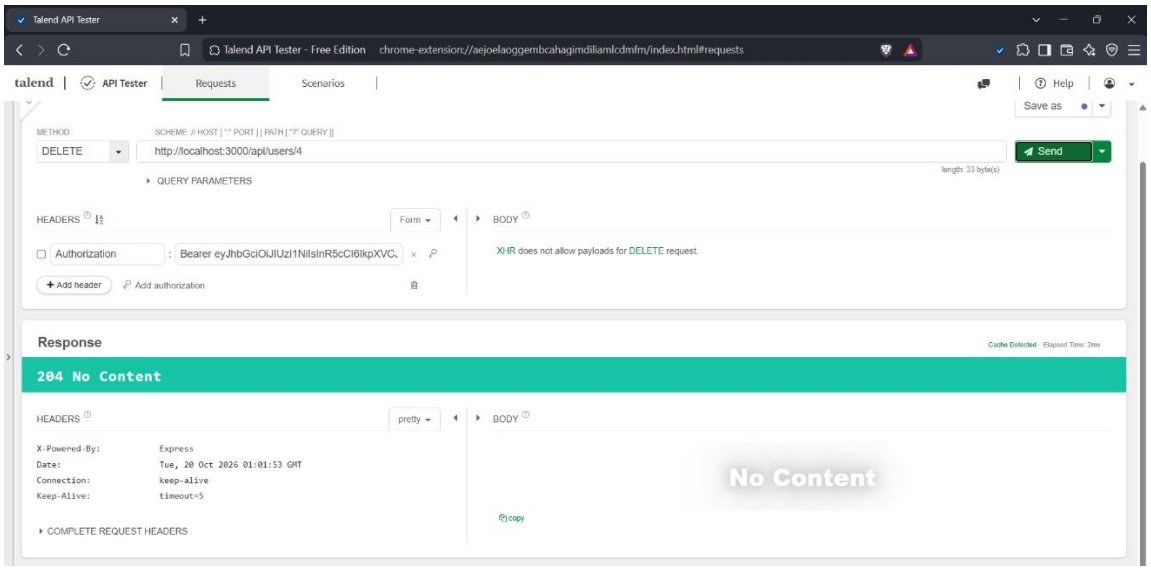
Deliverable: Screenshots of all 6 requests

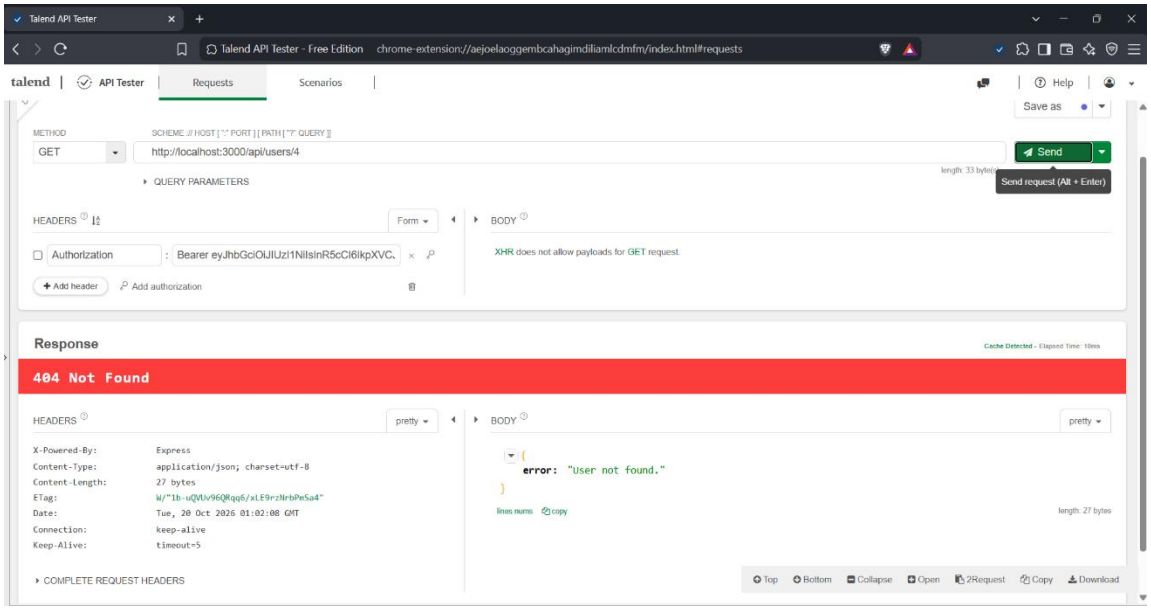
Task 5 — Test Full CRUD in Postman (10 pts)

#	Method	URL	Expected
1	GET	/api/users/1	200
2	PUT	/api/users/4	200 (name and role updated)
3	DELETE	/api/users/4	204 No Content
4	GET	/api/users/4	404





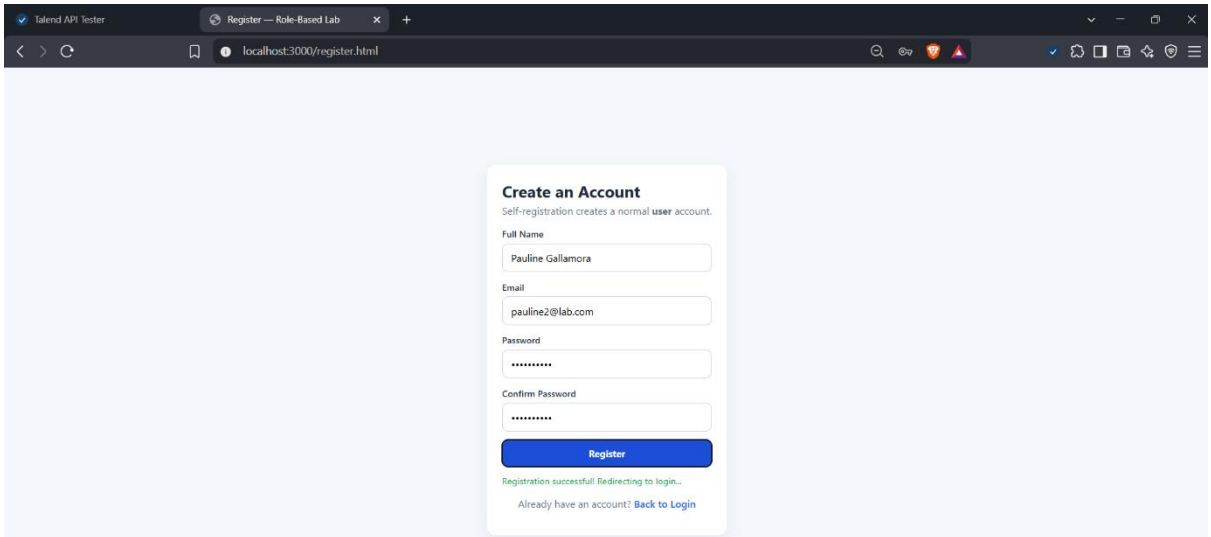


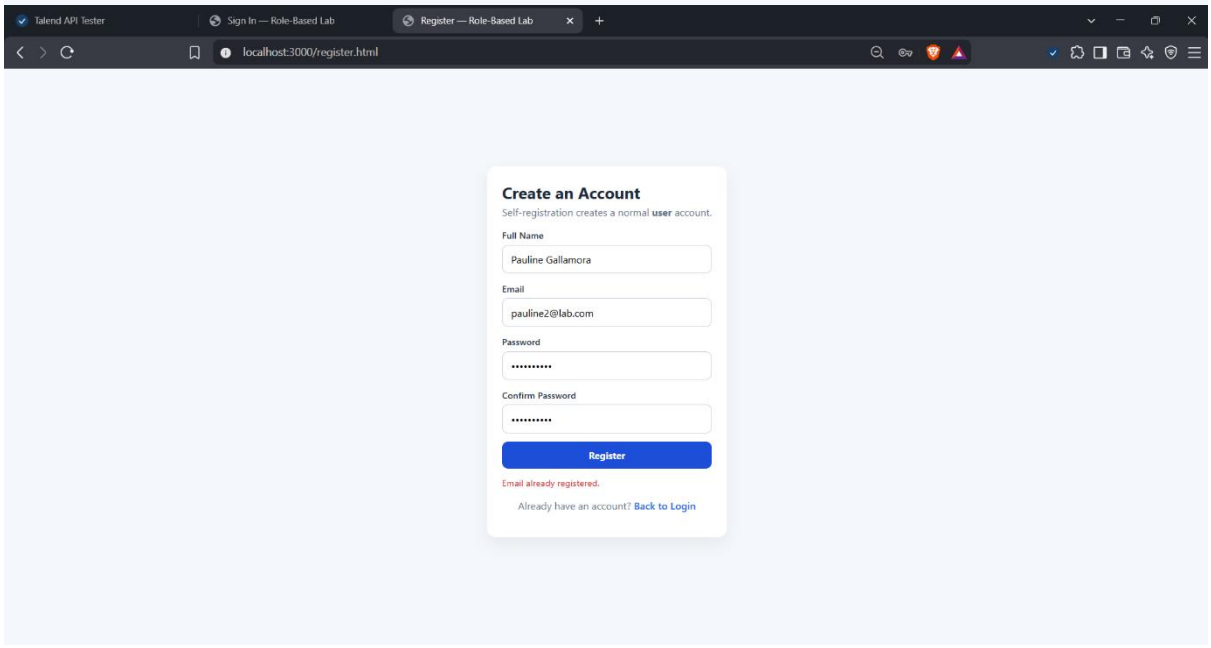


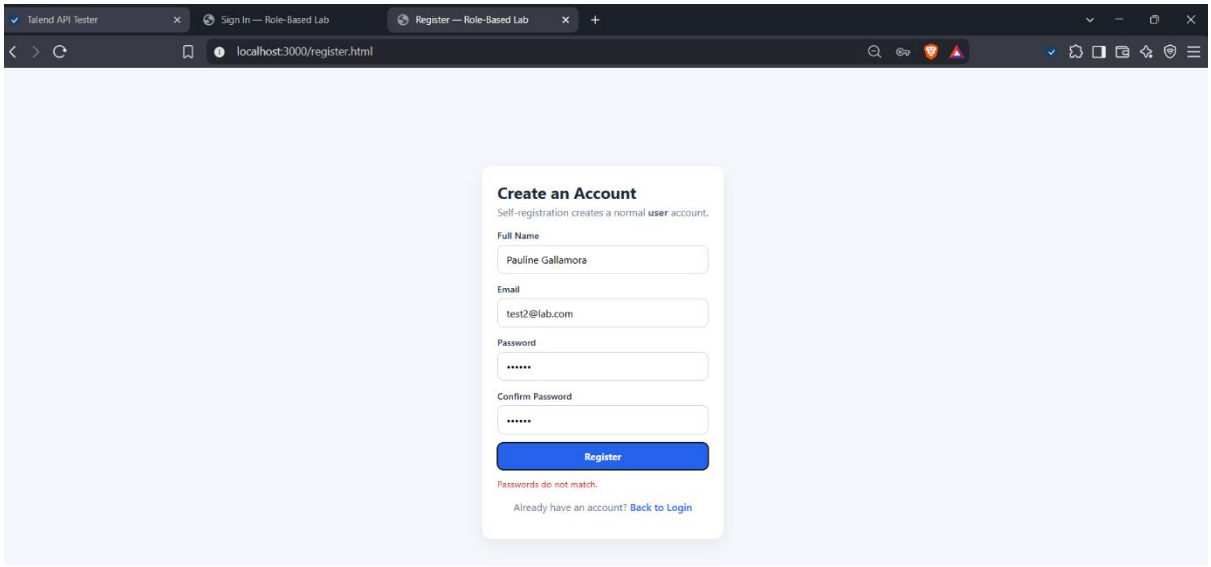
Deliverable: Screenshots of all 4 requests.

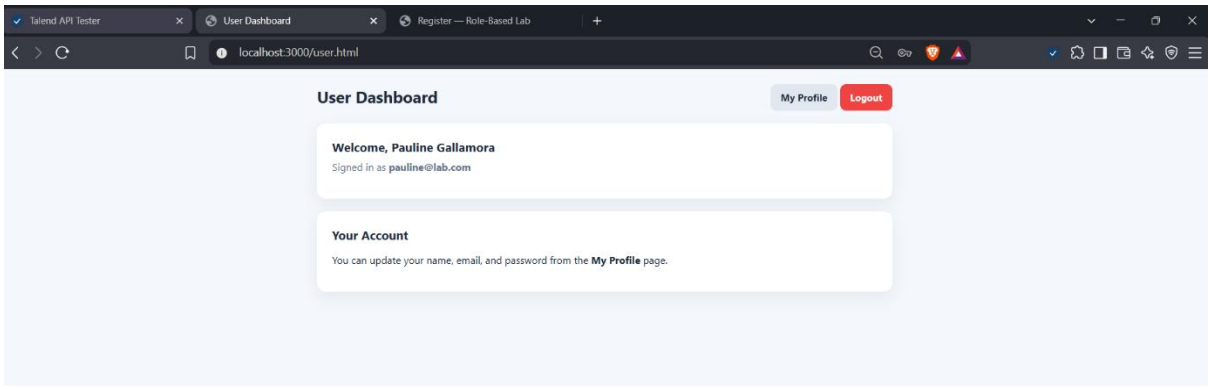
Task 6 — Browser Registration Flow (10 pts)

I registered a new account on register.html, then tried the same email again and mismatched passwords, and finally logged in with the new account.





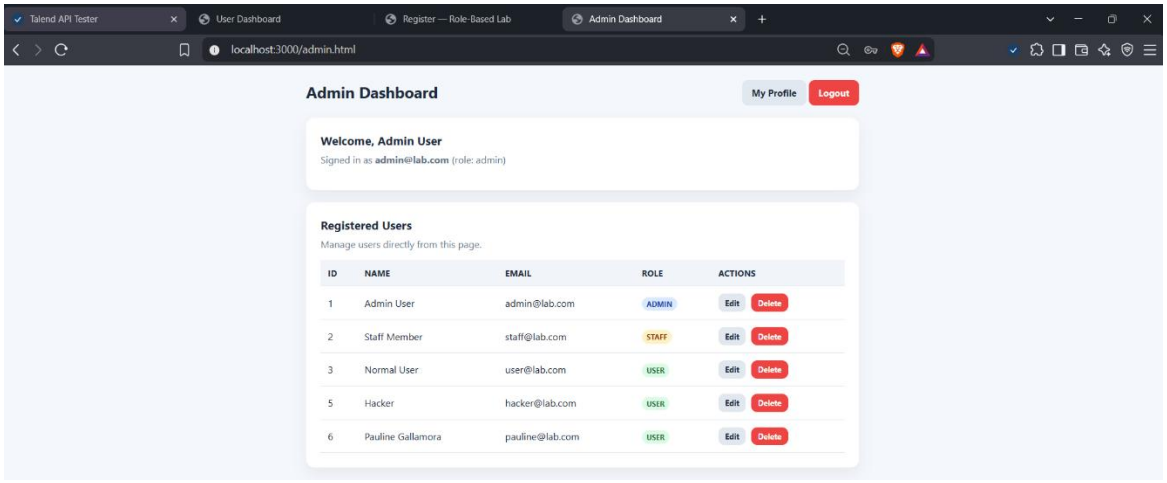


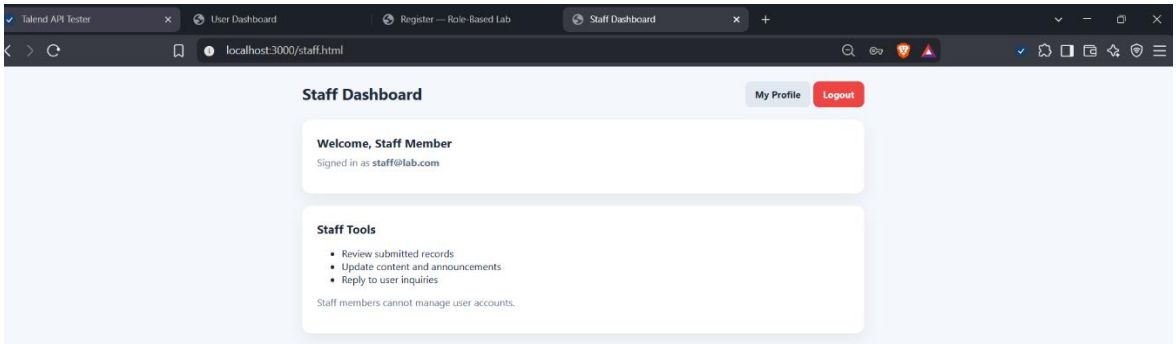


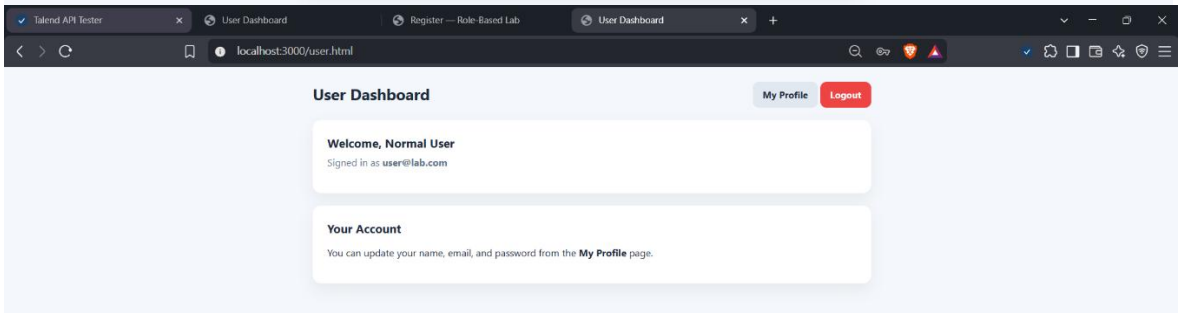
Deliverable: Screenshots of successful registration, duplicate email, mismatched password, and user dashboard

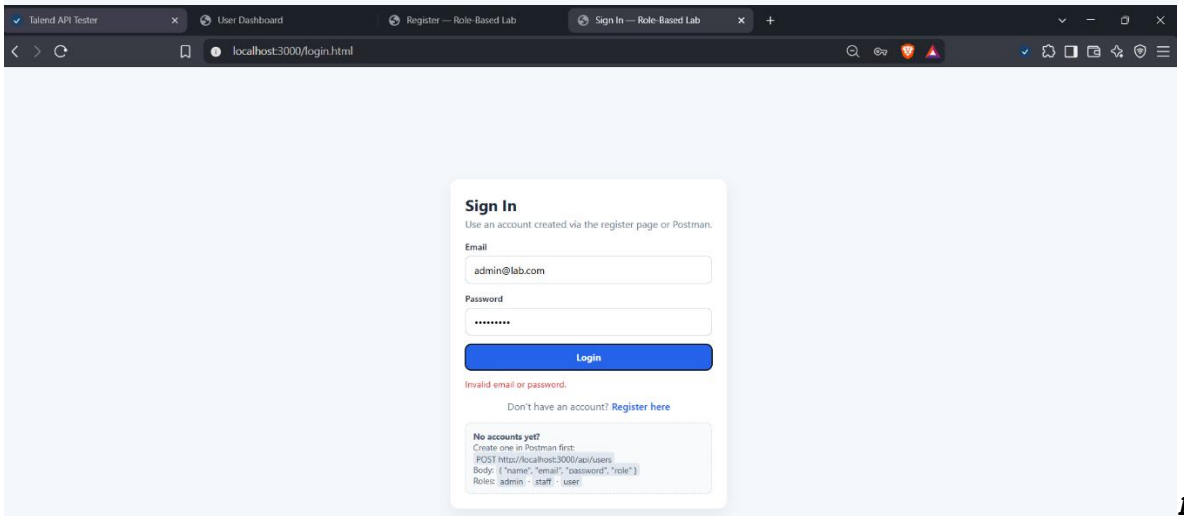
Task 7 — Browser Login & Role-Based Redirects (10 pts)

I logged in as each role and each one landed on its own page. A wrong password showed a red error.





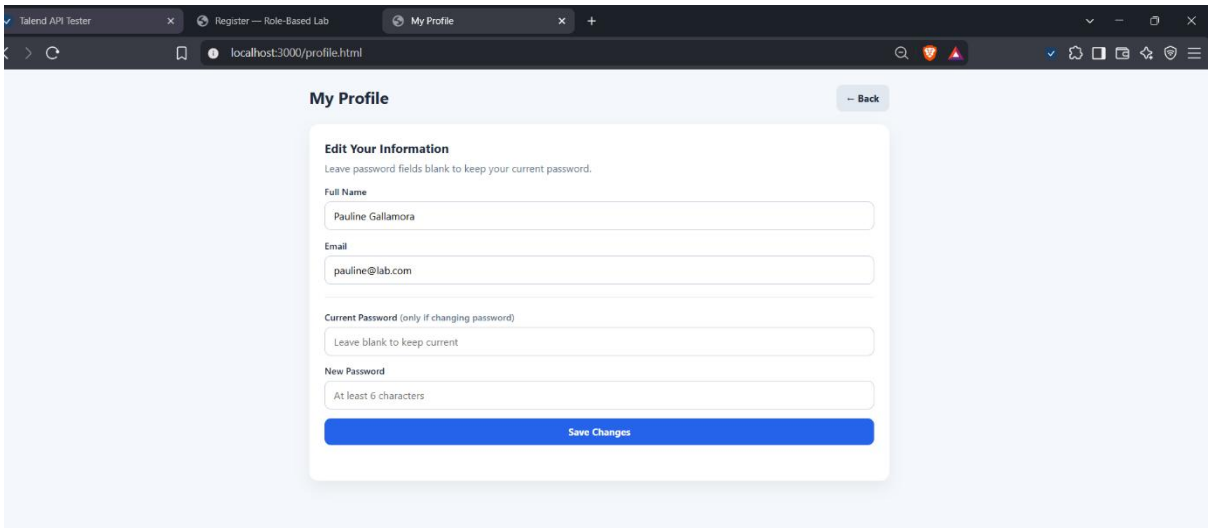


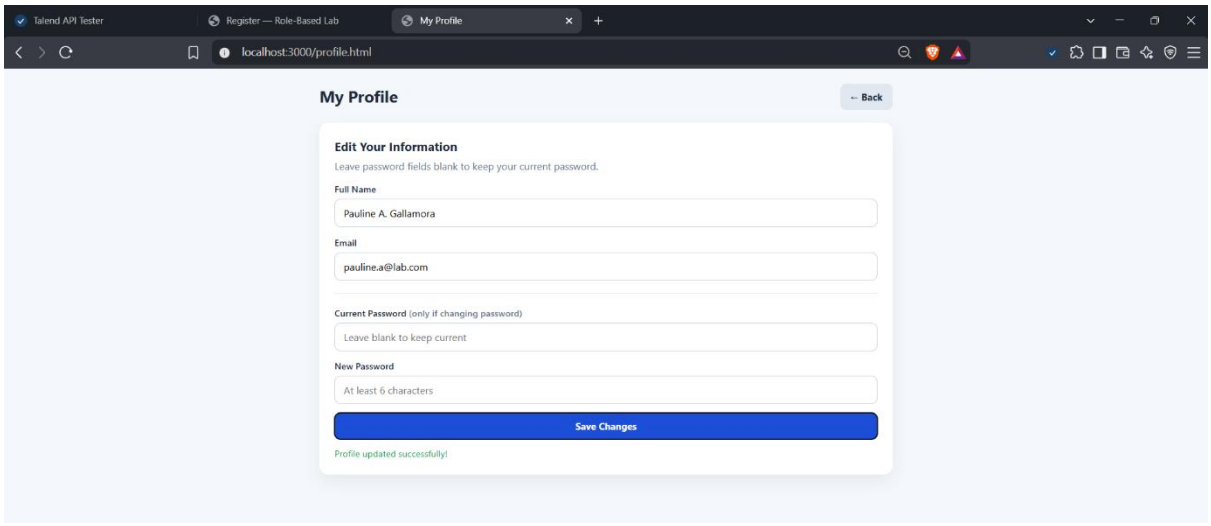


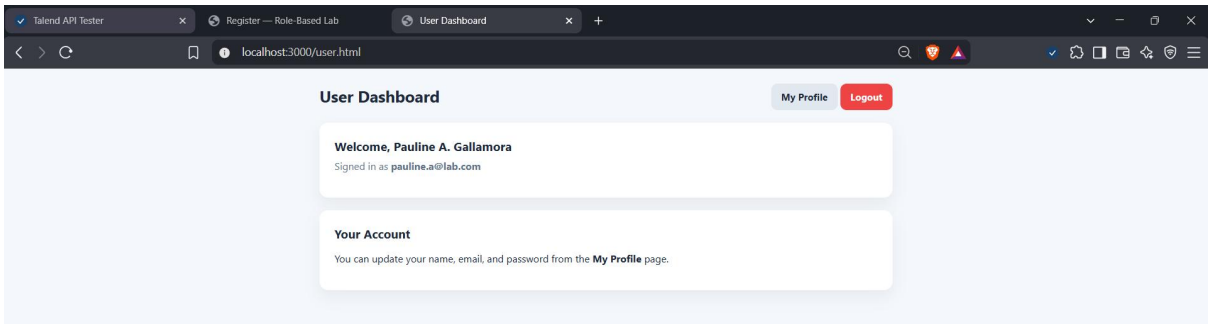
Deliverable: Screenshots of all three dashboards and the error.

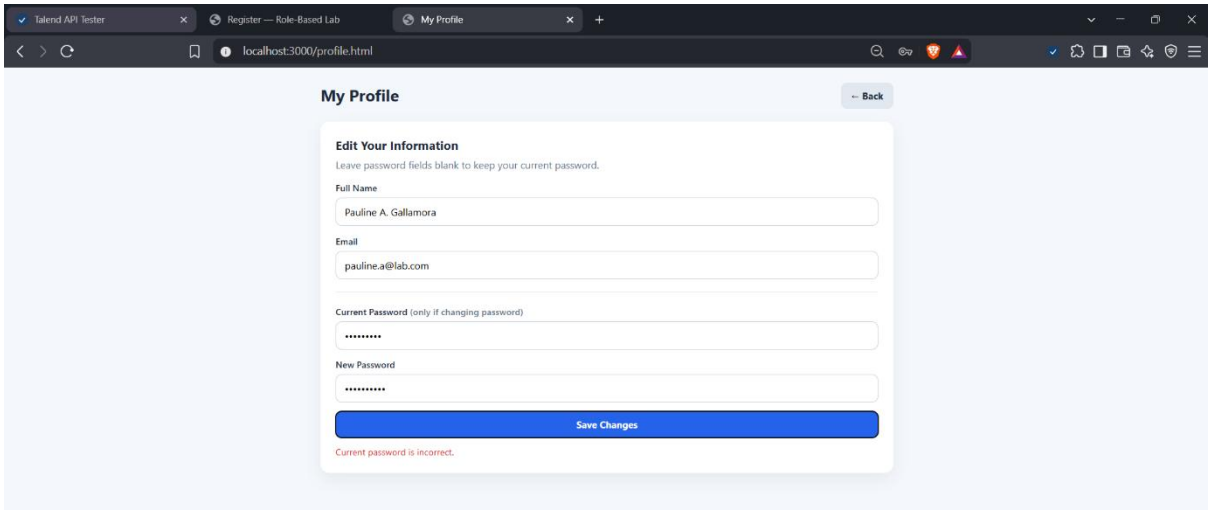
Task 8 — Profile Editing (10 pts)

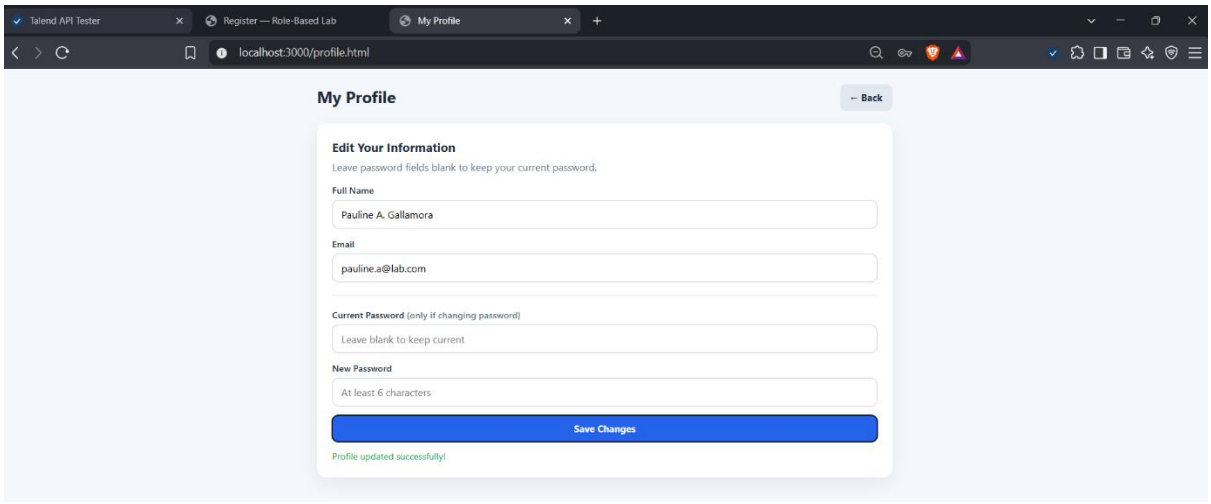
I logged in as my own account, opened My Profile, updated my name and email, tested a wrong current password, changed the password correctly, then logged in again with the new password.

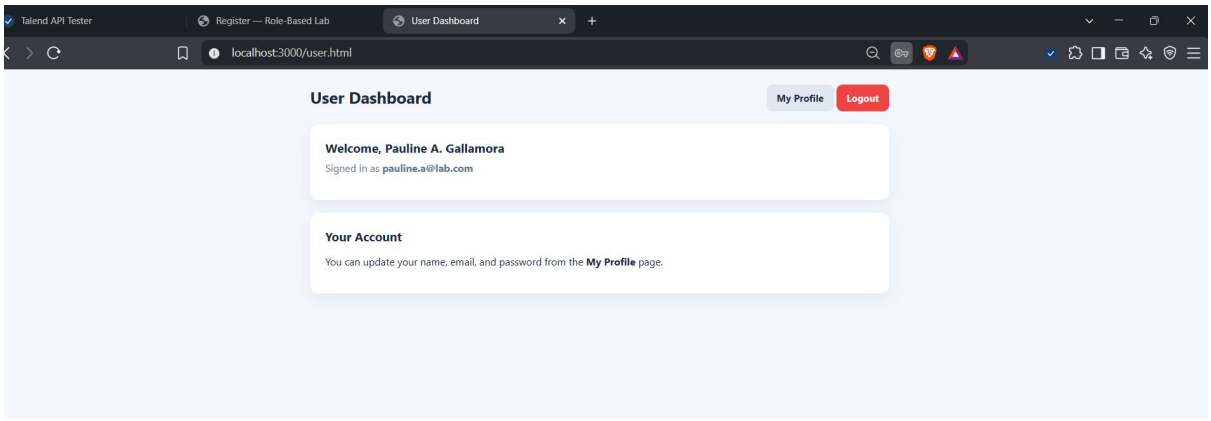








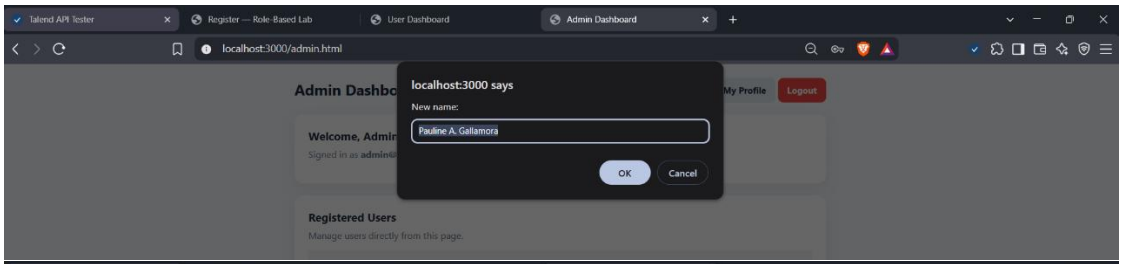


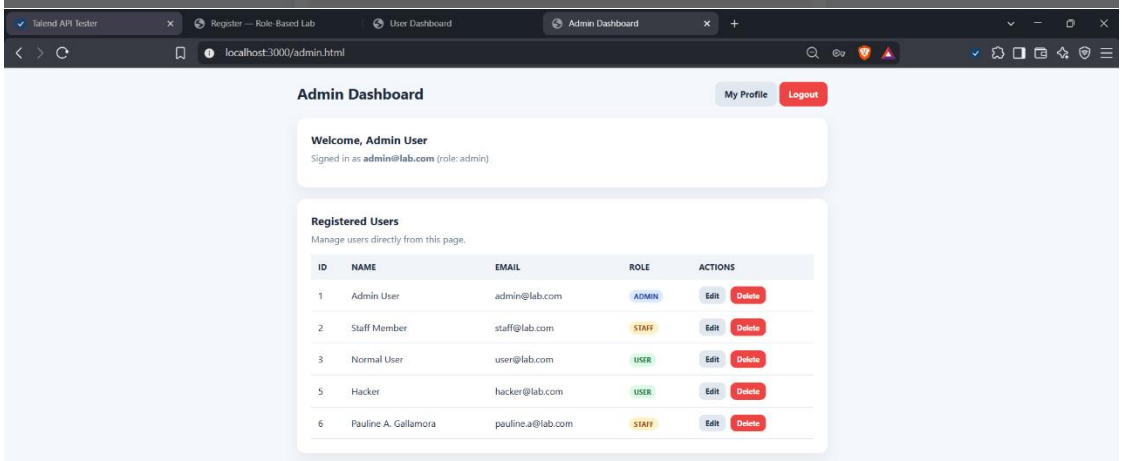


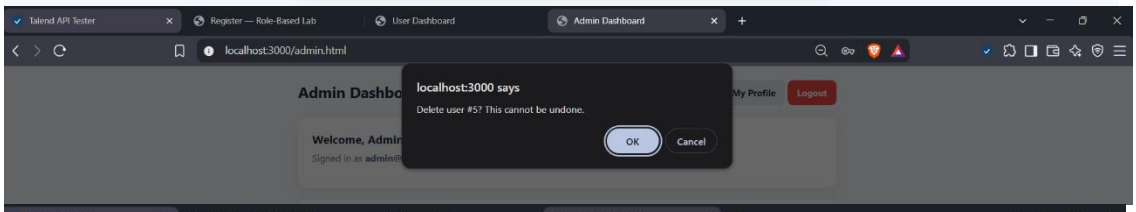
Deliverable: Screenshots of: profile page, successful update, wrong-password error, login with new password

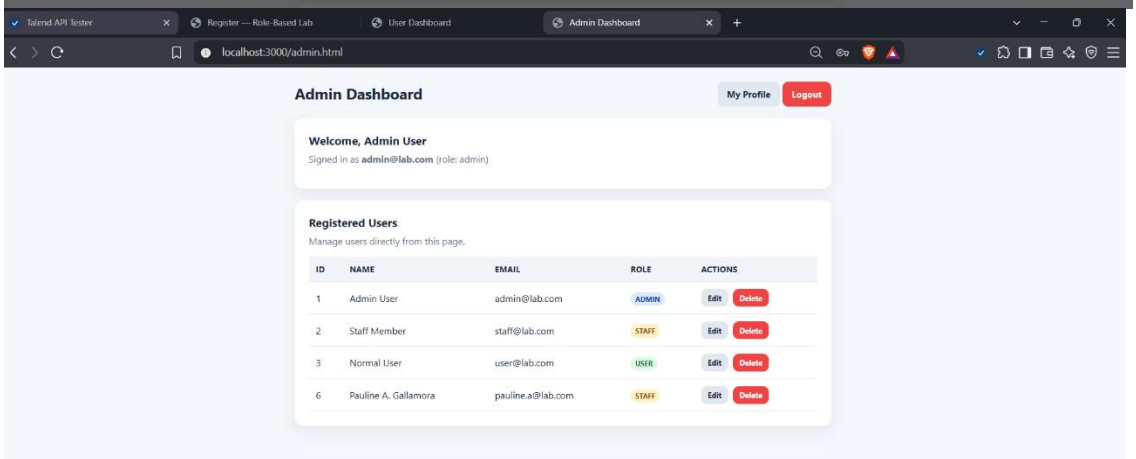
Task 9 — Admin User Management (10 pts)

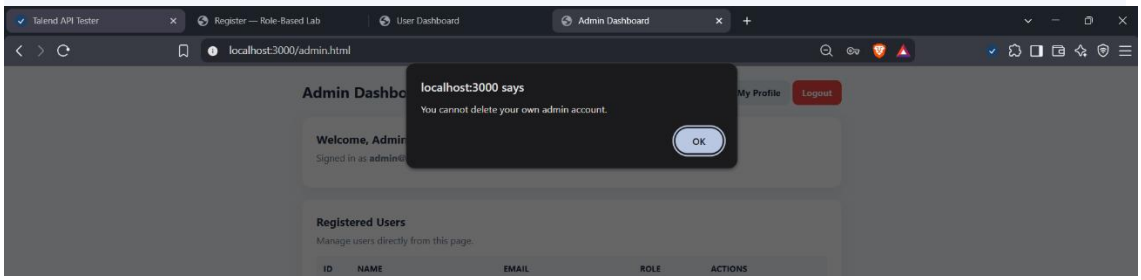
As admin, I edited a user (promoted to staff), deleted a user, and tried to delete my own admin account, which was blocked.











Deliverable: Screenshots of edit, delete, and blocked self delete.

Task 10 — Reflection Questions (10 pts)

Q1. In server.js, paste the exact block that decides redirectTo.

```
let redirectTo = '/user.html';
if (user.role === 'admin') redirectTo = '/admin.html';
else if (user.role === 'staff') redirectTo = '/staff.html';
```

Q2. What happens if a user manually sets sessionStorage.currentUser = {"role":"admin"} in DevTools? What does this reveal about client-side guards?

- **What happens:** The admin page only checks sessionStorage, so it will open and show the admin layout even if the person never logged in as admin.
- **What it shows:** Client-side guards only control what the page shows. They are not real security, because the user controls the browser. In this lab the API routes also have no server-side check, so the real protection has to be done on the server.

Q3. Why is the role forced to 'user' in /api/register?

- **Reason:** Registration is open to anyone. If the server accepted any role from the form, a person could register themselves as admin by changing the request.
- **So:** New accounts always start as normal users, and only an admin can create or promote other roles.

Q4. Why does /api/profile/:id require the current password before accepting a new one?

- **Reason:** It proves the person changing the password is the real owner of the account and not someone who just got on their open browser.
- **Also:** It stops a stranger from locking the owner out of their own account.

Q5. Why does sanitizeUser strip passwords from every response?

- **Reason:** Passwords should never leave the server. If they were in the JSON, anyone could see them in the browser's network tab or in Postman, and every list of users would leak everyone's passwords.

Q6. What happens to registered users when the server restarts? How is this fixed in real applications?

- **What happens:** They are all lost, because they are only stored in the users array in memory.
- **Fix:** Real applications store users in a database, so the data stays when the server restarts.

Q7. How does this lab demonstrate synchronous client–server communication?

- **Answer:** The browser sends a request, such as a login, and waits for the server's response before doing the next thing. For example, the login page does not move to the dashboard until the server replies with the redirectTo value. It is one request, one response, in order.

Q8. What role does JSON play in every request and response?

- **Answer:** JSON is the common format both sides understand. The browser or Postman sends data as JSON (like the email and password), and the server answers in JSON (like the user info or an error message). It makes it easy for different programs to share the same data.

Q9. Give one example of authentication and one of authorization in this lab.

- **Authentication:** Logging in with an email and password. The server checks them to confirm who the user is.
- **Authorization:** Deciding what a role is allowed to do or see, like only admins getting the user management table, or the server blocking the admin from deleting their own account.

Q10. Propose one security improvement for production. Explain why.

- **Improvement:** Hash the passwords with bcrypt and use real session cookies or tokens that the server checks on every request.
- **Why:** Right now passwords are stored in plain text and the role checks are only in the browser. Hashing protects passwords if the data leaks, and server-side checks make sure that people cannot fake being an admin.

Task 11 — Testing Checklist (5 pts)

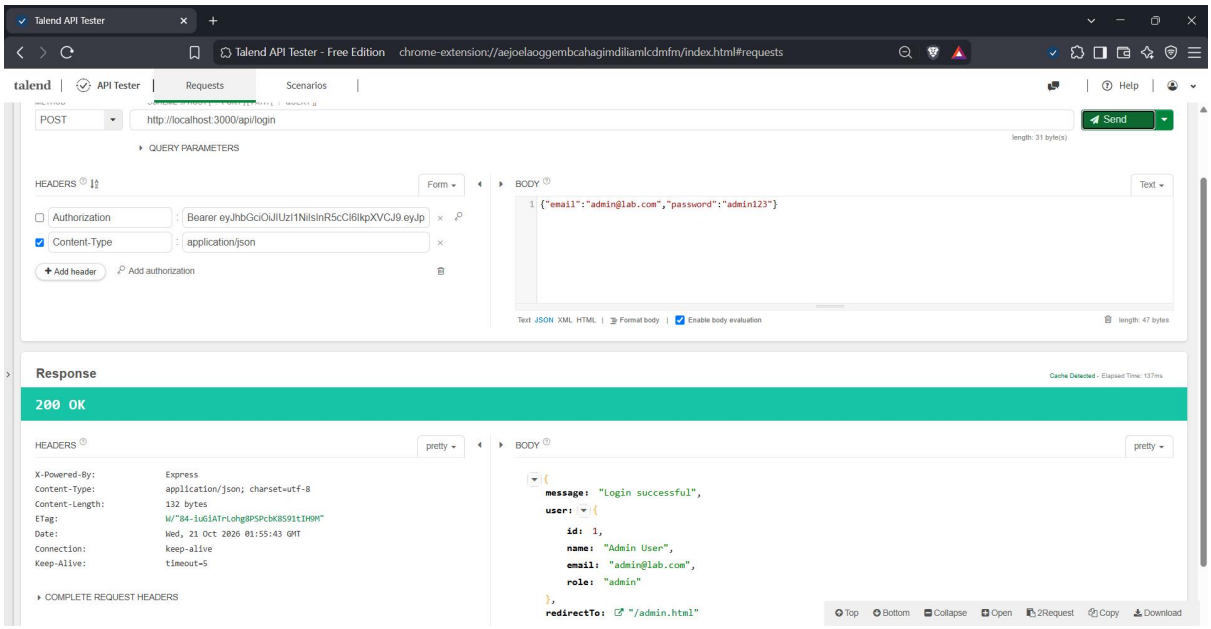
#	Test Case	Expected	Done
1	Server starts with node server.js	Console shows URLs	✓
2	Login page loads with styling	Renders correctly	✓
3	Register page loads	Renders correctly	✓
4	GET /api/users when empty	[]	✓
5	POST /api/users creates admin	201, no password	✓
6	POST /api/users creates staff	201, role=staff	✓
7	POST /api/users creates user	201, role=user	✓
8	GET /api/users	No passwords shown	✓
9	POST /api/register valid	201, role=user	✓
10	POST /api/register duplicate	409	✓
11	POST /api/register short password	400	✓
12	POST /api/register with role=admin	Role forced to user	✓
13	POST /api/login admin	200, redirectTo=/admin.html	✓
14	POST /api/login staff	200, redirectTo=/staff.html	✓
15	POST /api/login user	200, redirectTo=/user.html	✓
16	POST /api/login wrong password	401	✓
17	POST /api/login unknown email	401 (same error)	✓
18	POST /api/login missing password	400	✓
19	Browser register → login → user dashboard	Works	✓
20	Browser login as admin	Redirects to /admin.html	✓

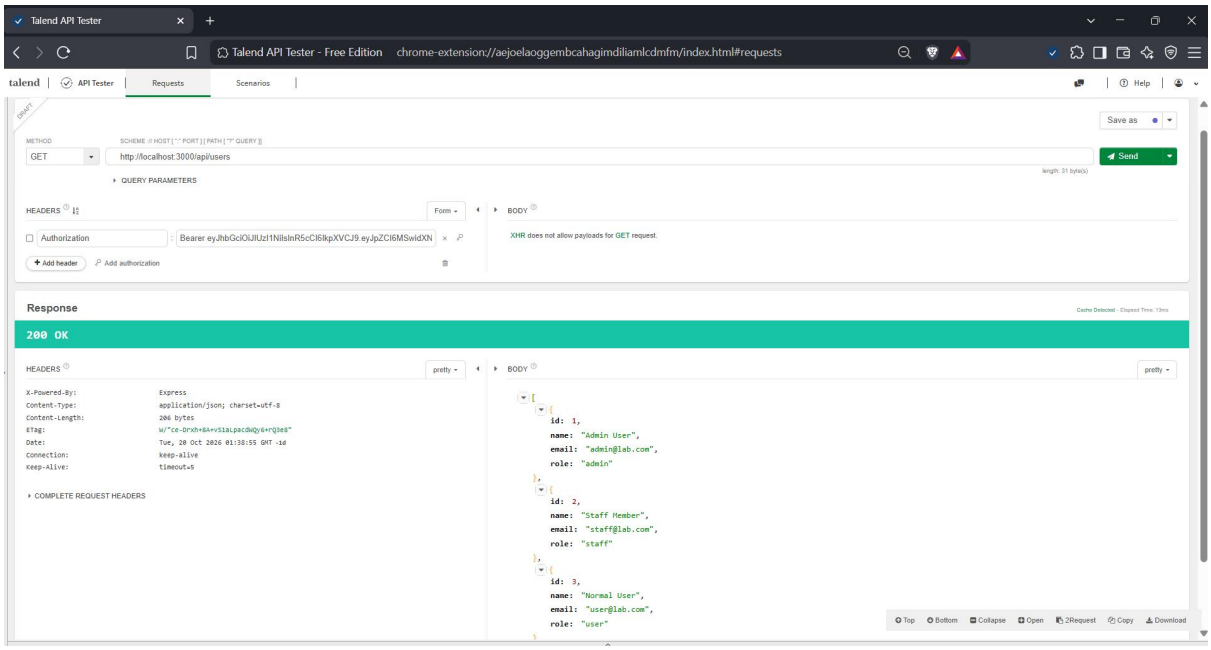
21	Browser login as staff	Redirects to /staff.html	✓
22	Browser login as user	Redirects to /user.html	✓
23	Logout clears sessionStorage	Returns to login	✓
24	Profile page loads current data	Pre-filled	✓
25	PUT /api/profile/:id name/email	200	✓
26	PUT profile with wrong current password	401	✓
27	PUT profile with correct current password	200	✓
28	Login with new password	200	✓
29	Admin edit user	Table updates	✓
30	Admin delete user	Row removed	✓
31	Admin delete own account	Blocked	✓
32	GET deleted user	404	✓
33	Restart server — all users cleared	Login fails	✓

IX. Bonus Task (+10 pts) — Hash Passwords with bcrypt

I installed bcrypt and updated server.js so passwords are hashed before they are stored (in POST /api/users, POST /api/register, PUT /api/users/:id and PUT /api/profile/:id). Login and the current-password check now use bcrypt.compare. After restarting the server, I recreated the users in Talend. Login still works, and GET /api/users does not show any passwords or hashes.

```
const hashed = await bcrypt.hash(password, 10);
const match = await bcrypt.compare(password, user.password);
```





Deliverable: Updated server.js (included in the submitted files) and screenshots showing login still works and GET /api/users never reveals hashes.

Notes and Observations

- At first, POST /api/register returned 404 Not Found. The cause was that the wrong or old server was running, which had no register route. I stopped it, ran node server.js from the correct project folder, and it worked.
- The first run of node server.js said "Cannot find module 'express'". I fixed it by running npm install inside the project folder.
- Because the users are only in memory, every restart emptied the list, so I had to recreate the users in Task 2 and Task 3 after restarting.
- For the bonus, I installed bcrypt with npm install bcrypt, replaced the plain-text password checks with bcrypt.hash and bcrypt.compare, and recreated all users after restarting the server.
- The routes /api/users and /api/users/:id have no server-side check, so anyone who knows the URL can call them. Only the pages check the role in the browser.